

Python analysis documentation: sample ROI-specific thickness and neutron attenuation coefficient calculations

version 2.0, August 17, 2026

1 Overview

This document describes the three Python modules that make up the neutron attenuation analysis workflow. The shared objective is to determine the effective macroscopic cross-section (Σ) of complex 3D samples (solder joints, wires and gilding) from Monte Carlo simulated neutron transmission data, and to compare those values against the ranges derived from neutron computed tomography (nCT).

Two independent estimators of Σ are implemented, and a third module presents the final comparison.

Module 1 — `python_ray_casting.py`

Geometry-aware Σ . STL ray casting, followed by a ray-by-ray Beer–Lambert forward model and root finding on the transmission.

Module 2 — `calculate_neutron_attenuation_coefficients.py`

Detector-response Σ . A single effective thickness L and the open/sample count ratio, with optional Gd-157 efficiency weighting.

Module 3 — `plot_neutron_attenuation_coefficients.py`

Presentation. A hard-coded summary of the final Σ_{MC} values plotted against the nCT-derived bands.

The essential distinction between the first two modules is the treatment of thickness. The ray casting module uses the *full distribution* of path lengths through the mesh, so it correctly handles samples whose thickness varies across the beam footprint. The second module collapses the sample to a *single* representative thickness L and is therefore only valid where the ROI is close to uniform; it is, however, the only module that accounts for the energy-dependent response of the detector.

2 Prerequisites

The following Python dependencies are required across the three modules:

- `numpy`, `scipy`: vector mathematics, interpolation (`interp1d`), root finding (Brent’s method) and nearest-neighbour search (`KDTree`).

- `trimesh`, `pyvista`: STL geometry handling, ray casting and 3D mesh rendering.
- `uproot`: reading output `.root` files from the Geant4 simulations.
- `matplotlib`: 2D thickness projections, spectra and statistical histograms.
- Standard library: `multiprocessing`, `functools`, `os`, `math`.

All simulation input is read from ROOT files through the branch `NeutronEnergyData/NeutronKE_MeV`, which holds the per-neutron kinetic energy in MeV at the scoring plane. Both analysis modules assume this branch exists in both the open beam and the sample file. File paths in all three scripts are placeholders (`/path_to/...`) and must be edited before running.

3 Module 1

`python_ray_casting.py`

This module determines Σ from first principles by combining the true geometric thickness distribution of the sample with the simulated transmission.

3.1 Configuration

The user block at the top of the script selects the sample and its geometry:

- `SAMPLE_CHOICE`: one of `'solder'`, `'wire'` or `'gilding'`. Each branch sets `INPUT_STL`, `OPEN_BEAM_FILE`, `RESULTS_FILE` and `OUTPUT_PREFIX`.
- `RAY_RESOLUTION` (default 512): the $N \times N$ ray grid.
- `ROTATE_MESH`, `ROTATION_AXIS` (default `[0, 0, 1]`) and `ROTATION_ANGLE_DEG`: the transform that aligns the mesh with the simulated beam path (270° for the solder joint, 180° for the wire and gilding).
- `ROI_SIZE_{X,Y,Z}_MM` and `ROI_CENTER_{X,Y,Z}_MM`: the axis-aligned region of interest, matching the active detection area of the simulation. Alternative ROI definitions (e.g. the 0.3 mm and top-of-sample boxes for the solder joint) are retained as commented blocks.
- `MATERIAL_RANGES`: the nCT-derived Σ bounds per material, used only to report the expected transmission window as a sanity check.

3.2 Geometric initialisation and ray casting

`generate_thickness_maps()` loads the STL, applies the rotation, and builds a uniform grid of parallel rays spanning the mesh bounding box in Y and Z . Ray origins are placed one unit beyond the maximum X bound and the direction vector is $\hat{\mathbf{d}} = \langle -1, 0, 0 \rangle$, so the beam propagates along $-X$.

Intersections are evaluated in `_calculate_object_thickness_for_chunk()` via `mesh.ray.intersects_location(..., multiple_hits=True)`. For each ray, hits are

sorted by distance from the origin and paired as successive entry/exit points, so that the material thickness for ray i is

$$x_i = \sum_{j=0,2,4,\dots} \|\mathbf{p}_{j+1} - \mathbf{p}_j\|$$

Only rays with an even hit count ≥ 2 are accepted; odd counts indicate degenerate grazing or non-watertight intersections and are discarded (this also means internal voids are handled correctly, as the empty span between an exit and the next entry is never accumulated). Rays that miss the mesh remain NaN and are masked out.

The ray grid is split into contiguous chunks and dispatched over a `multiprocessing.Pool` using `cpu_count()-1` workers by default; each chunk carries its global start index so results can be reassembled in order. Trimesh works in the STL's native units (metres for these files), so all lengths are multiplied by 1000 to give millimetres on return.

3.3 Definition of the region of interest

The full thickness map $\chi = \{x_1, \dots, x_M\}$ is filtered to the ROI by a mask on the Y and Z coordinates of the nominal surface location (the midpoint of the first and last hit of each ray), yielding χ_{ROI} . The X extent of the ROI is not used for filtering, since rays already traverse the full object along X ; it only defines the wireframe box drawn in the 3D render.

3.4 Attenuation model and root finding

The forward model, `ray_by_ray_transmission()`, evaluates the Beer-Lambert law per ray and averages over the ROI:

$$T_{\text{theory}}(\Sigma) = \frac{1}{K} \sum_{i=1}^K \exp(-\Sigma \cdot x_i)$$

where K is the number of valid rays in the ROI and x_i is expressed in cm. Averaging the exponentials rather than exponentiating the mean thickness is what makes this estimator robust to thickness variation across the ROI.

`calculate_simulation_results()` loads the two ROOT files and forms the empirical transmission from the raw detected counts:

$$T_{\text{sim}} = \frac{N_{\text{sample}}}{N_{\text{open}}}$$

The effective cross-section is then the root of

$$f(\Sigma) = T_{\text{theory}}(\Sigma) - T_{\text{sim}} = 0$$

solved with Brent's method (`scipy.optimize.brentq`) over the bracket $[0.001, 20.0] \text{ cm}^{-1}$, which spans physically realistic thermal and cold neutron values. The solve is attempted only when $0 < T_{\text{sim}} < 1$; a failure to bracket the root is caught and reported rather than raised. For context the function also prints the transmission window implied by the nCT bounds in `MATERIAL_RANGES`, noting that the lower Σ bound corresponds to the higher transmission.

3.5 Visualisation and verification

- **2D projections** (`plot_roi_projections`): XZ and YZ scatter plots coloured by thickness on a shared `viridis` normalisation. The YZ view overlays the ROI as a dashed red rectangle; the XZ view, where the ROI is not bounded in X , instead draws the two Z limits as horizontal lines.
- **Statistical profiles** (`plot_thickness_statistics`): a histogram of χ_{ROI} with mean, median and interquartile range overlaid, and the same statistics plus standard deviation and min/max printed to stdout. Output is written as SVG (the PNG call is currently commented out).
- **3D render** (`render_3d_thickness_map`): a KDTree maps each mesh vertex to its nearest ray intersection point, accepting matches within 1.0 mm, and the resulting per-vertex thickness is rendered in PyVista with the ROI drawn as a red wireframe box. Runs off-screen by default and saves both a screenshot and a vector graphic.

3.6 Execution

The `__main__` block checks that the STL exists, generates the thickness maps, calls the ROOT analysis, and then produces the projections, statistics and 3D render. Colour limits for the projections and the render use the full-object thickness range, while the histogram is scaled to the ROI range.

4 Module 2

`calculate_neutron_attenuation_coefficients.py`

This module provides an independent estimate of Σ from the count ratio alone, and quantifies the effect of the energy-dependent detector response.

4.1 Configuration

`SAMPLE_CHOICE` again selects the open beam and sample ROOT files. The `sample_thicknesses` dictionary supplies a single representative path length L in cm per sample (wire 0.04121, solder 0.08664, gilding 0.01635; an alternative solder value of 0.04703 is retained as a comment). The plotting range is fixed by $X_AXIS_MIN_MeV = 10^{-9}$ and $X_AXIS_MAX_MeV = 10$, and spectra are binned on 100 logarithmic bins across that interval.

4.2 Detector efficiency model

`load_cross_section_data()` reads the tabulated Gd-157 radiative capture cross-section from `Gd_cross_sec.txt` (columns: energy in eV, cross-section in barns; lines beginning with `[` are treated as comments).

`get_gd_efficiency_interpolator()` builds the detector response. The raw efficiency is taken to be proportional to the capture cross-section, the Gd-157 enrichment of the screen and the optical light-collection efficiency,

$$\varepsilon_{\text{raw}}(E) = \sigma_{\text{capt}}(E) \cdot f_{\text{enr}} \cdot f_{\text{opt}}$$

which is then normalised to its maximum,

$$\varepsilon(E) = \frac{\varepsilon_{\text{raw}}(E)}{\max_E \varepsilon_{\text{raw}}(E)}$$

and wrapped in a linear `interp1d` over energy in eV. Out-of-range behaviour is asymmetric by design: below the tabulated range the first efficiency value is held constant, while above it the efficiency is set to zero, so fast neutrons carry no weight.

In the `__main__` block the two factors are set to $f_{\text{enr}} = 0.8817$ (enrichment in the screen) and $f_{\text{opt}} = 0.70$ (light output above 70%).

Note on the scaling factors. Because the efficiency is normalised by its own maximum, f_{enr} and f_{opt} cancel exactly and have no effect on either the normalised curve or the derived $\Sigma_{\text{corrected}}$. They are constant multipliers on both the open beam and the sample sums, and therefore vanish in the ratio. Only the *shape* of $\sigma_{\text{capt}}(E)$ matters. The script relies on this explicitly when it re-calls the interpolator with dummy factors of 1.0 for plotting. The factors should be understood as documentation of the physical setup rather than as parameters that influence the result.

4.3 Macroscopic coefficient calculation

`calculate_macroscopic_coefficient()` loads both ROOT files and computes two variants of the coefficient from

$$\Sigma = \frac{1}{L} \ln \left(\frac{I_{\text{open}}}{I_{\text{sample}}} \right)$$

- **Uncorrected:** I is the raw number of detected neutrons, i.e. every neutron is weighted equally regardless of energy.
- **Corrected:** energies are converted from MeV to eV and each neutron is weighted by $\varepsilon(E)$ before summing,

$$I = \sum_k \varepsilon(E_k)$$

so the result reflects what a Gd-157 screen would actually register.

Both values are printed alongside the corresponding counts. If the ROOT files cannot be opened the arrays fall back to empty and the routine continues so that the efficiency and cross-section plots can still be produced.

Validity. Unlike Module 1, this estimator assumes a single uniform path length L through the sample. It is appropriate for near-uniform ROIs and for isolating the efficiency-weighting effect, but for strongly varying geometry the ray-by-ray result of Module 1 should be preferred, since $\langle e^{-\Sigma x} \rangle \neq e^{-\Sigma \langle x \rangle}$.

4.4 Outputs

Two figures are written:

- `neutron_attenuation_analysis.png`: a two-panel figure. The upper panel shows the normalised Gd-157 efficiency on the left axis and the true capture cross-section in barns on a twinned right axis (both logarithmic); the lower panel shows the incident and attenuated neutron spectra as step histograms on the shared logarithmic energy axis.
- `neutron_spectra_only.png`: the spectra panel alone, with an explicit log–log grid, for use as a standalone figure.

The function returns the corrected and uncorrected coefficients, the efficiency energy grid, the cross-section array and the plot filename.

5 Module 3

`plot_neutron_attenuation_coefficients.py`

This module is presentation-only: it contains no analysis and reads no data files. It renders the final comparison figure from values transcribed by hand from the runs of Modules 1 and 2.

5.1 Data

Ten measurement locations are defined: the wire, two solder joint ROIs (ROI 1 and ROI 2) and seven gilding ROIs. For each, `sim_values` holds the simulated Σ_{MC} in cm^{-1} and `sim_errors` the run-to-run sample standard deviation over the ten Monte Carlo runs. The nCT-derived ranges are given as intervals — wire (2.3, 2.5), solder joint (1.9, 2.1), gilding (3.1, 3.8) — each with a 1σ uncertainty on the bound itself (0.02, 0.02 and 0.17 cm^{-1} respectively, the latter consistent with the inversion).

5.2 Figure construction

`generate_attenuation_plot()` draws, for each material group, a hatched grey `Rectangle` spanning the nCT range over the relevant x positions, plus a fainter band of half-width σ centred on each bound. Simulated values are overlaid as error-bar markers, keyed by group (circle for wire, diamond for solder joint, cross for gilding) and coloured black, orangered and dodgerblue. The legend is assembled explicitly from proxy artists so that the two range patches appear once each rather than once per group. Styling is centralised in the `CONFIG` dictionary (font sizes, tick geometry, line widths, DPI).

5.3 Outputs

The figure is saved at 300 DPI as `neutron_attenuation_coeff.png` and, with a transparent background, as `neutron_attenuation_coeff.svg`. Note that the `savepath` argument in the function signature is unused; the output paths are hard-coded in the body and must be edited directly. The function is called at import time at the bottom of the file, so importing the module will produce the figure as a side effect.

6 Suggested order of use

1. Run Module 1 once per sample and ROI to obtain the geometry-corrected Σ , together with the thickness maps and diagnostic plots used to confirm the ROI is placed correctly.
2. Run Module 2 to obtain the corrected and uncorrected count-ratio coefficients and the spectral figures, as a cross-check on the detector response.
3. Transcribe the accepted Σ_{MC} values and their run-to-run spreads into Module 3 and regenerate the summary figure.

7 Known limitations

- All file paths are placeholders and must be set before running any module.
- Module 2 uses a single scalar thickness per sample and does not share the ROI definition or the thickness distribution used by Module 1; the two are consistent only where the ROI is close to uniform.
- The enrichment and optical efficiency factors in Module 2 cancel under normalisation and do not affect the reported coefficient (see above).
- Module 3 holds its inputs as literals, so any change to the analysis requires the values to be updated by hand; there is no automated link back to Modules 1 and 2.
- Neither analysis module propagates statistical uncertainty on Σ from the Poisson counting error; the error bars in Module 3 come from repeated Monte Carlo runs rather than from a per-run calculation.